SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

SLE-4: 8-Puzzle Solver Using BFS and DFS

PRN: 25UAM109

Name: Pratiksha Jagadish Khondre

Division: B

Date: 10/10/2026

Status: Accepted

PART A: Architecture Decision Record (ADR)

Title: Choosing BFS and DFS for empirical performance comparison in the 8-Puzzle Solver

1. Context

The 8-Puzzle is a search problem in which numbered tiles must be moved into a goal arrangement. The solver needs to explore possible puzzle states and find a path from the initial state to the goal. Different search algorithms can explore states in different orders and use different amounts of time and memory. In my SLE work, I implemented Breadth-First Search (BFS) and Depth-First Search (DFS), then measured their performance on the same solvable input. A visited-state set helps prevent the search from repeatedly exploring states that have already been seen.

2. Decision

I chose to implement both Breadth-First Search (BFS) and Depth-First Search (DFS) to compare their performance on the 8-Puzzle. I used the same solvable initial and goal states for a fair comparison; visited-state tracking was used to avoid repeated exploration.

3. Alternatives Considered

- Alternative 1: Use only BFS. It finds a shortest solution when each move has equal cost, but does not provide a comparison with another search strategy.
- Alternative 2: Use only DFS. It can explore deeply, but may spend time exploring a long path before finding a solution and does not guarantee the shortest solution.
- Alternative 3: Do not track visited states
  This was rejected because the solver could revisit the same puzzle arrangements through different move sequences, causing unnecessary work and potentially making the search continue for much longer.

4. Rationale / Justification

The visited set helps the solver remember puzzle arrangements that have already been handled. This reduces repeated exploration and keeps the search process controlled.

The same design decision is useful for both algorithms, although BFS and DFS explore states in different orders. BFS uses a queue to explore states level by level, while DFS uses a stack-like order to explore deeper paths first. Visited-state tracking helps both strategies avoid unnecessary revisits.

In SLE-2, I compared BFS and DFS on the same 8-puzzle input and measured execution time and states expanded. For the input 7 2 4 / 5 0 6 / 8 3 1, BFS took an average of 124.331 ms and expanded 55,409 states, while DFS took an average of 251.130 ms and expanded 99,900 states. These measurements show that the two strategies explored different numbers of states and had different running times for this test case. The visited-state mechanism supports controlled exploration, while the search strategy also affects the results.

This decision is consistent with the SLE-3 C4 architecture, where the search system includes state generation and visited/memory handling alongside the search engine.

5. Consequences

Positive:

- BFS and DFS can be compared using measured time and states expanded.
- Visited-state tracking avoids repeated exploration of known states.
- Separate modules make the system easier to test, explain and maintain.
- Avoids unnecessary repeated processing of the same puzzle state.
- Keeps search exploration more controlled.
- Supports meaningful counting of states expanded during SLE-2 performance testing.

Negative / Trade-offs:

- BFS can require a large amount of memory because it stores many states at the same search depth.
- DFS may explore many unnecessary states and does not guarantee the shortest solution.
- Performance depends on the initial puzzle state; these measurements cannot represent every possible puzzle.

PART B: Short Summary of My SLE Journey

SLE-1

In SLE-1, I created a simple rule-based chatbot. I learned the basics of building a small AI-related program and documenting the code and AI contribution.

SLE-2

In SLE-2, I worked on an 8-Puzzle Solver using BFS and DFS. I compared the algorithms using execution-time measurements and the number of states expanded. I tested both algorithms on the same input so their results could be compared.

For the test input 7 2 4 / 5 0 6 / 8 3 1, the recorded average time was 124.331 ms for BFS and 251.130 ms for DFS. BFS expanded 55,409 states and DFS expanded 99,900 states.

SLE-3

In SLE-3, I prepared the C4 architecture for the 8-Puzzle Solver. The design describes how the user provides the initial puzzle, how the system generates possible next states, how the search engine applies BFS or DFS, and how the result is presented. State tracking is an important part of controlling the search.

SLE-4

In SLE-4, I selected the use of a visited set as an important design decision in my BFS and DFS implementation. I documented the context, decision, alternatives, rationale, and consequences in an Architecture Decision Record.

Part C — AI Contribution

AI tools were used to help understand the SLE-4 requirements, explain the design decision, organize the ADR sections, and improve the clarity of the report. I selected the 8-Puzzle Solver topic, reviewed the implementation and performance results, and used the project results to explain the decision.

What AI helped with: Explaining BFS/DFS concepts, suggesting code and documentation improvements, organizing the ADR, and preparing practice viva questions.

What I did myself: I selected the 8-Puzzle topic, ran and tested the code, compared the measured results, reviewed the project structure and diagrams, and checked the final content against my SLE work.

Conclusion

The important design decision in my project is to track visited puzzle states during BFS and DFS. This helps avoid unnecessary repeated exploration and makes the search process more controlled. The SLE-2 measurements helped me compare the two search strategies, while SLE-3 provided the architecture in which state tracking and search are represented. SLE-4 helped me document and justify this implementation decision.